

# Algorithm and Data Structure

## Part 5: Greedy Algorithms

Ali El Hadi Ismail Fawaz

ENSISA 1A Info, Université de Haute-Alsace



Une école d'ingénieurs de l'Université de Haute-Alsace



# Table of Contents

- 1 Overview
- 2 What is a Greedy Algorithm
- 3 Easy Problems
- 4 Medium Problems
- 5 Advanced Problems

# Overview

# Overview

- **What is a Greedy Algorithm?**

- Core idea: make the locally optimal choice at each step.
- When this strategy works and when it fails.

- **Key Concepts**

- Greedy-choice property
- Optimal substructure

- **Easy real-life examples**

- **First simple problem**

- **Why it can work**

- **When it can fail**

## What is a Greedy Algorithm

# What is a Greedy Algorithm?

- A method that builds a solution step by step
- At each step, choose the option that looks best **right now**
- No backtracking, once a choice is made, it's final
- Works well for some problems, but not for all

# What is a Greedy Algorithm?

- A method that builds a solution step by step
- At each step, choose the option that looks best **right now**
- No backtracking, once a choice is made, it's final
- Works well for some problems, but not for all

## Real-life example:

Always take the largest coin first when making change.



# When This Strategy Works and When It Fails

- **When it works:**

- The problem has the *greedy-choice property*, choosing the best now leads to the best overall
- The problem has *optimal substructure*, optimal solutions are built from optimal subproblems
- Examples: Activity selection, Huffman coding, Minimum spanning tree.

- **When it fails:**

- The best local choice does not lead to the best overall solution
- Lacks greedy-choice property or optimal substructure
- Example: Coin change with certain denominations



## Easy Problems

# Smallest Subset Sum Greater Than All Others

- **Problem statement:** Given an array of non-negative integers, find the *fewest elements* whose sum is greater than the sum of the remaining elements
- **Example:**
  - Input:  $[3, 1, 7, 1]$
  - Output:  $[7]$  — since  $7 > 1 + 3 + 1 = 5$
- **Naive algorithm:**
  - Enumerate all subsets; pick the valid one with minimum size
  - **Complexity:**  $O(2^n \cdot n)$  exponential

# Smallest Subset Sum Greater Than All Others

- **Problem statement:** Given an array of non-negative integers, find the *fewest elements* whose sum is greater than the sum of the remaining elements
- **Example:**
  - Input: [3, 1, 7, 1]
  - Output: [7] — since  $7 > 1 + 3 + 1 = 5$
- **Naive algorithm:**
  - Enumerate all subsets; pick the valid one with minimum size
  - **Complexity:**  $O(2^n \cdot n)$  exponential
- **Greedy strategy:**
  - Sort descending; keep taking the largest elements
  - Stop when chosen sum  $>$  total/2.
  - **Complexity:**  $O(n \log n)$  overall
- **Why greedy works:** Big numbers get you over the finish line with fewer steps
- **Application:** Budgeting, pick the few largest expenses whose total is more than half of the total budget

# Smallest Subset Sum Greater Than All Others

## Greedy Algorithm

Input: Array  $A$ , size  $n$

Output: size of smallest subset with bigger sum than all others

Instructions:

1.  $total \leftarrow sum(A)$
2. sort  $A$  in descending order
3.  $count \leftarrow 0$
4.  $current\_total \leftarrow 0$
5. For  $i$  from 0 to  $n-1$  do
  - a.  $current\_total \leftarrow current\_total + A[i]$
  - b.  $count \leftarrow count + 1$
  - c. If  $current\_total > total - current\_total$   
Return  $count$
6. Return  $count$

# Maximize Sum of Consecutive Differences (Circular Array)

- **Problem statement:** Given a circular array, rearrange its elements to maximize  $\sum_{i=1}^n |a_i - a_{i+1}|$ , where  $a_{n+1} = a_1$
- **Example:**
  - Input: [4, 2, 1, 8]
  - Output: 18 , Rearrange given array as : [1, 8, 2, 4]
  - $\sum_{i=1}^n |a_i - a_{i+1}| = |1 - 8| + |8 - 2| + |2 - 4| + |4 - 1| = 7 + 6 + 2 + 3 = 18$

# Maximize Sum of Consecutive Differences (Circular Array)

- **Problem statement:** Given a circular array, rearrange its elements to maximize  $\sum_{i=1}^n |a_i - a_{i+1}|$ , where  $a_{n+1} = a_1$
- **Example:**
  - Input: [4, 2, 1, 8]
  - Output: 18 , Rearrange given array as : [1, 8, 2, 4]
  - $\sum_{i=1}^n |a_i - a_{i+1}| = |1 - 8| + |8 - 2| + |2 - 4| + |4 - 1| = 7 + 6 + 2 + 3 = 18$
- **Naive approach:** Generate all permutations, compute the sum for each, and take the maximum. **Complexity:**  $O(n! \cdot n)$  — infeasible for large  $n$

# Maximize Sum of Consecutive Differences (Circular Array)

- **Problem statement:** Given a circular array, rearrange its elements to maximize  $\sum_{i=1}^n |a_i - a_{i+1}|$ , where  $a_{n+1} = a_1$
- **Example:**
  - Input: [4, 2, 1, 8]
  - Output: 18 , Rearrange given array as : [1, 8, 2, 4]
  - $\sum_{i=1}^n |a_i - a_{i+1}| = |1 - 8| + |8 - 2| + |2 - 4| + |4 - 1| = 7 + 6 + 2 + 3 = 18$
- **Naive approach:** Generate all permutations, compute the sum for each, and take the maximum. **Complexity:**  $O(n! \cdot n)$  — infeasible for large  $n$
- **Greedy idea:** Sort the array, then alternate smallest, largest, next smallest, next largest, etc. Example pattern:  $a_1, a_n, a_2, a_{n-1}, \dots$  **Complexity:** Sorting  $O(n \log n)$  + arrangement  $O(n)$

# Maximize Sum of Consecutive Differences (Circular Array)

- **Problem statement:** Given a circular array, rearrange its elements to maximize  $\sum_{i=1}^n |a_i - a_{i+1}|$ , where  $a_{n+1} = a_1$
- **Example:**
  - Input: [4, 2, 1, 8]
  - Output: 18 , Rearrange given array as : [1, 8, 2, 4]
  - $\sum_{i=1}^n |a_i - a_{i+1}| = |1 - 8| + |8 - 2| + |2 - 4| + |4 - 1| = 7 + 6 + 2 + 3 = 18$
- **Naive approach:** Generate all permutations, compute the sum for each, and take the maximum. **Complexity:**  $O(n! \cdot n)$  — infeasible for large  $n$
- **Greedy idea:** Sort the array, then alternate smallest, largest, next smallest, next largest, etc. Example pattern:  $a_1, a_n, a_2, a_{n-1}, \dots$  **Complexity:** Sorting  $O(n \log n)$  + arrangement  $O(n)$
- **Why it works:** Brings elements with the largest differences next to each other, maximizing total sum
- **Application:** Arrange sensors around a circular track so that very high and very low readings are next to each other, maximizing variation



# Maximize Sum of Consecutive Differences (Circular Array)

Unsorted array

|    |   |   |   |    |   |    |   |
|----|---|---|---|----|---|----|---|
| 12 | 3 | 7 | 0 | -4 | 9 | -1 | 5 |
|----|---|---|---|----|---|----|---|

# Maximize Sum of Consecutive Differences (Circular Array)

Unsorted array

|    |   |   |   |    |   |    |   |
|----|---|---|---|----|---|----|---|
| 12 | 3 | 7 | 0 | -4 | 9 | -1 | 5 |
|----|---|---|---|----|---|----|---|

Sorted array

|    |    |   |   |   |   |   |    |
|----|----|---|---|---|---|---|----|
| -4 | -1 | 0 | 3 | 5 | 7 | 9 | 12 |
|----|----|---|---|---|---|---|----|

# Maximize Sum of Consecutive Differences (Circular Array)

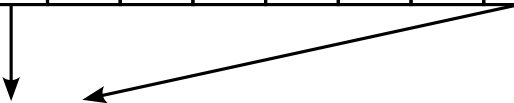
Unsorted array

|    |   |   |   |    |   |    |   |
|----|---|---|---|----|---|----|---|
| 12 | 3 | 7 | 0 | -4 | 9 | -1 | 5 |
|----|---|---|---|----|---|----|---|

Sorted array

|    |    |   |   |   |   |   |    |
|----|----|---|---|---|---|---|----|
| -4 | -1 | 0 | 3 | 5 | 7 | 9 | 12 |
|----|----|---|---|---|---|---|----|

|    |    |  |  |  |  |  |  |
|----|----|--|--|--|--|--|--|
| -4 | 12 |  |  |  |  |  |  |
|----|----|--|--|--|--|--|--|



# Maximize Sum of Consecutive Differences (Circular Array)

Unsorted array

|    |   |   |   |    |   |    |   |
|----|---|---|---|----|---|----|---|
| 12 | 3 | 7 | 0 | -4 | 9 | -1 | 5 |
|----|---|---|---|----|---|----|---|

Sorted array

|    |    |   |   |   |   |   |    |
|----|----|---|---|---|---|---|----|
| -4 | -1 | 0 | 3 | 5 | 7 | 9 | 12 |
|----|----|---|---|---|---|---|----|



|    |    |    |  |  |  |  |  |
|----|----|----|--|--|--|--|--|
| -4 | 12 | -1 |  |  |  |  |  |
|----|----|----|--|--|--|--|--|

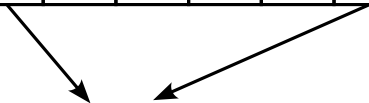
# Maximize Sum of Consecutive Differences (Circular Array)

Unsorted array

|    |   |   |   |    |   |    |   |
|----|---|---|---|----|---|----|---|
| 12 | 3 | 7 | 0 | -4 | 9 | -1 | 5 |
|----|---|---|---|----|---|----|---|

Sorted array

|    |    |   |   |   |   |   |    |
|----|----|---|---|---|---|---|----|
| -4 | -1 | 0 | 3 | 5 | 7 | 9 | 12 |
|----|----|---|---|---|---|---|----|



|    |    |    |   |  |  |  |  |
|----|----|----|---|--|--|--|--|
| -4 | 12 | -1 | 9 |  |  |  |  |
|----|----|----|---|--|--|--|--|

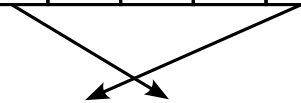
# Maximize Sum of Consecutive Differences (Circular Array)

Unsorted array

|    |   |   |   |    |   |    |   |
|----|---|---|---|----|---|----|---|
| 12 | 3 | 7 | 0 | -4 | 9 | -1 | 5 |
|----|---|---|---|----|---|----|---|

Sorted array

|    |    |   |   |   |   |   |    |
|----|----|---|---|---|---|---|----|
| -4 | -1 | 0 | 3 | 5 | 7 | 9 | 12 |
|----|----|---|---|---|---|---|----|



|    |    |    |   |   |  |  |  |
|----|----|----|---|---|--|--|--|
| -4 | 12 | -1 | 9 | 0 |  |  |  |
|----|----|----|---|---|--|--|--|

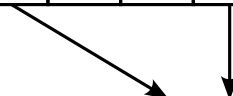
# Maximize Sum of Consecutive Differences (Circular Array)

Unsorted array

|    |   |   |   |    |   |    |   |
|----|---|---|---|----|---|----|---|
| 12 | 3 | 7 | 0 | -4 | 9 | -1 | 5 |
|----|---|---|---|----|---|----|---|

Sorted array

|    |    |   |   |   |   |   |    |
|----|----|---|---|---|---|---|----|
| -4 | -1 | 0 | 3 | 5 | 7 | 9 | 12 |
|----|----|---|---|---|---|---|----|



|    |    |    |   |   |   |  |  |
|----|----|----|---|---|---|--|--|
| -4 | 12 | -1 | 9 | 0 | 7 |  |  |
|----|----|----|---|---|---|--|--|

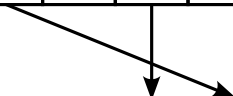
# Maximize Sum of Consecutive Differences (Circular Array)

Unsorted array

|    |   |   |   |    |   |    |   |
|----|---|---|---|----|---|----|---|
| 12 | 3 | 7 | 0 | -4 | 9 | -1 | 5 |
|----|---|---|---|----|---|----|---|

Sorted array

|    |    |   |   |   |   |   |    |
|----|----|---|---|---|---|---|----|
| -4 | -1 | 0 | 3 | 5 | 7 | 9 | 12 |
|----|----|---|---|---|---|---|----|



|    |    |    |   |   |   |   |  |
|----|----|----|---|---|---|---|--|
| -4 | 12 | -1 | 9 | 0 | 7 | 3 |  |
|----|----|----|---|---|---|---|--|



# Maximize Sum of Consecutive Differences (Circular Array)

Unsorted array

|    |   |   |   |    |   |    |   |
|----|---|---|---|----|---|----|---|
| 12 | 3 | 7 | 0 | -4 | 9 | -1 | 5 |
|----|---|---|---|----|---|----|---|

Sorted array

|    |    |   |   |   |   |   |    |
|----|----|---|---|---|---|---|----|
| -4 | -1 | 0 | 3 | 5 | 7 | 9 | 12 |
|----|----|---|---|---|---|---|----|

|    |    |    |   |   |   |   |   |
|----|----|----|---|---|---|---|---|
| -4 | 12 | -1 | 9 | 0 | 7 | 3 | 5 |
|----|----|----|---|---|---|---|---|



# Maximize Sum of Consecutive Differences (Circular Array)

Unsorted array

|    |   |   |   |    |   |    |   |
|----|---|---|---|----|---|----|---|
| 12 | 3 | 7 | 0 | -4 | 9 | -1 | 5 |
|----|---|---|---|----|---|----|---|

Sorted array

|    |    |   |   |   |   |   |    |
|----|----|---|---|---|---|---|----|
| -4 | -1 | 0 | 3 | 5 | 7 | 9 | 12 |
|----|----|---|---|---|---|---|----|

|    |    |    |   |   |   |   |   |
|----|----|----|---|---|---|---|---|
| -4 | 12 | -1 | 9 | 0 | 7 | 3 | 5 |
|----|----|----|---|---|---|---|---|

Sum= 70

# Maximize Sum of Consecutive Differences (Circular Array)

## Greedy Algorithm

Input: Array  $A$ , size  $n$

Output: Maximize Sum of Consecutive Differences

Instructions:

1. sort  $A$  in increasing order
2.  $count \leftarrow 0$
3. For  $i$  from 0 to  $n/2 - 1$  do
  - a.  $count \leftarrow count + 2 * (A[n - 1 - i] - A[i])$
4. Return  $count$

- **Time Complexity:**  $\mathcal{O}(n \log n)$
- **Space Complexity:**  $\mathcal{O}(1)$ .
- Other solution to store new array has same **Time Complexity**  $\mathcal{O}(n \log n)$  but **Space Complexity** will becomes  $\mathcal{O}(n)$

## Medium Problems

# Minimum Number of Jumps to Reach End

- **Problem statement:** Given array  $a$ , where  $a[i]$  is the max jump from index  $i$ , find the minimum number of jumps to reach the last index (return  $-1$  if unreachable)
- **Example:**
  - Input:  $[1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9]$
  - Output: 3
  - Explanation: Path  $0 \rightarrow 1 \rightarrow 4 \rightarrow 10$  (from  $a[0]=1$  to 1;  $a[1]=3$  to 4;  $a[4]=9$  to end)

# Minimum Number of Jumps to Reach End

- **Problem statement:** Given array  $a$ , where  $a[i]$  is the max jump from index  $i$ , find the minimum number of jumps to reach the last index (return  $-1$  if unreachable)
- **Example:**
  - Input:  $[1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9]$
  - Output: 3
  - Explanation: Path  $0 \rightarrow 1 \rightarrow 4 \rightarrow 10$  (from  $a[0]=1$  to 1;  $a[1]=3$  to 4;  $a[4]=9$  to end)
- **Naive (brute force):** Explore all jump sequences, Time Complexity exponential in  $n$  (ex:  $\mathcal{O}(\varphi^n)$  when each step can be 1 or 2); Space Complexity  $\mathcal{O}(n)$  recursion stack

# Minimum Number of Jumps to Reach End

- **Problem statement:** Given array  $a$ , where  $a[i]$  is the max jump from index  $i$ , find the minimum number of jumps to reach the last index (return  $-1$  if unreachable)
- **Example:**
  - Input:  $[1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9]$
  - Output: 3
  - Explanation: Path  $0 \rightarrow 1 \rightarrow 4 \rightarrow 10$  (from  $a[0]=1$  to 1;  $a[1]=3$  to 4;  $a[4]=9$  to end)
- **Naive (brute force):** Explore all jump sequences, Time Complexity exponential in  $n$  (ex:  $\mathcal{O}(\varphi^n)$  when each step can be 1 or 2); Space Complexity  $\mathcal{O}(n)$  recursion stack
- **Greedy (single pass):**
  - Loop over array through index  $i$  and track *farthest* reach and *current* reach (end of current jump range)
  - When position  $i$  is the same as current reach: take a jump, set current reach to farthest reach
  - If at any time farthest is less or equal to position  $i$  (which is not the end of the array) then result is unreachable  $\Rightarrow -1$

# Minimum Number of Jumps to Reach End

- **Problem statement:** Given array  $a$ , where  $a[i]$  is the max jump from index  $i$ , find the minimum number of jumps to reach the last index (return  $-1$  if unreachable)
- **Example:**
  - Input:  $[1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9]$
  - Output: 3
  - Explanation: Path  $0 \rightarrow 1 \rightarrow 4 \rightarrow 10$  (from  $a[0]=1$  to 1;  $a[1]=3$  to 4;  $a[4]=9$  to end)
- **Naive (brute force):** Explore all jump sequences, Time Complexity exponential in  $n$  (ex:  $\mathcal{O}(\varphi^n)$  when each step can be 1 or 2); Space Complexity  $\mathcal{O}(n)$  recursion stack
- **Greedy (single pass):**
  - Loop over array through index  $i$  and track *farthest* reach and *current* reach (end of current jump range)
  - When position  $i$  is the same as current reach: take a jump, set current reach to farthest reach
  - If at any time farthest is less or equal to position  $i$  (which is not the end of the array) then result is unreachable  $\Rightarrow -1$
- **Complexity (greedy):** Time Complexity  $\mathcal{O}(n)$  Space Complexity  $\mathcal{O}(1)$
- **Application:** Minimize relay “hops” along a line (routers, charging stations, transfer segments)



# Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

## Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

$i = 0$     $\text{jump} = 0$     $\text{maxReach} = 0$   
 $\text{currentReach} = 0$

## Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

$i = 0$     $\text{jump} = 0$     $\text{maxReach} = 1$   
 $\text{currentReach} = 0$

## Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

$i = 0$     $\text{jump} = 1$     $\text{maxReach} = 1$   
 $\text{currentReach} = 1$

# Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

$i = 1$     $\text{jump} = 1$     $\text{maxReach} = 4$   
 $\text{currentReach} = 1$

## Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

$i = 1$     $\text{jump} = 2$     $\text{maxReach} = 4$   
 $\text{currentReach} = 4$

## Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

$i = 2$     $\text{jump} = 2$     $\text{maxReach} = 4$   
 $\text{currentReach} = 4$

# Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

$i = 3$    jump = 2   maxReach = 5  
currentReach = 4



# Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

$i = 4$     $\text{jump} = 2$     $\text{maxReach} = 5$   
 $\text{currentReach} = 4$

# Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

$i = 4$     $\text{jump} = 3$     $\text{maxReach} = 5$   
 $\text{currentReach} = 5$

## Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

$i = 5$     $\text{jump} = 3$     $\text{maxReach} = 9$   
 $\text{currentReach} = 5$

# Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

$i = 5$    jump = 4   maxReach = 9  
currentReach = 9

# Minimum Number of Jumps to Reach End

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 2 | 1 | 4 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

$i = 5$     $\text{jump} = 4$     $\text{maxReach} = 9$   
 $\text{currentReach} = 9$

# Minimum Number of Jumps to Reach End

## Greedy Algorithm

Input: Array A, size n

Output: Minimum Number of Jumps to Reach End

Instructions:

1. If  $A[0] == 0$   
Return -1
2.  $jumps, maxReach, currentReach \leftarrow 0, 0, 0$
3. For  $i$  from 0 to  $n - 1$  do
  - a.  $maxReach \leftarrow \max(maxReach, i + A[i])$
  - b. If  $maxReach \geq n - 1$   
Return  $jump + 1$
  - c. If  $currentReach == i$   
If  $maxReach == i$   
Return -1  
Else  
 $jump \leftarrow jump + 1$   
 $currentReach \leftarrow maxReach$
4. Return -1

# Smallest Number with Given Digits and Digit Sum

- **Problem:** Given digits  $d$  and sum  $s$ , find the smallest  $d$ -digit number with digit-sum  $s$  (no leading zeros), or report  $-1$  if impossible
- **Example:**
  - Input:  $d = 3, s = 20$
  - Output: 299
  - Explanation:  $2 + 9 + 9 = 20$

# Smallest Number with Given Digits and Digit Sum

- **Problem:** Given digits  $d$  and sum  $s$ , find the smallest  $d$ -digit number with digit-sum  $s$  (no leading zeros), or report  $-1$  if impossible
- **Example:**
  - Input:  $d = 3, s = 20$
  - Output: 299
  - Explanation:  $2 + 9 + 9 = 20$
- **Naive (brute force):** Check all  $d$ -digit numbers (from  $10^{d-1}$  to  $10^d - 1$ ) and test their digit sums; **Time Complexity**  $\mathcal{O}(10^d \cdot d)$ , **Space Complexity**  $\mathcal{O}(1)$



# Smallest Number with Given Digits and Digit Sum

- **Problem:** Given digits  $d$  and sum  $s$ , find the smallest  $d$ -digit number with digit-sum  $s$  (no leading zeros), or report  $-1$  if impossible
- **Example:**
  - Input:  $d = 3, s = 20$
  - Output: 299
  - Explanation:  $2 + 9 + 9 = 20$
- **Naive (brute force):** Check all  $d$ -digit numbers (from  $10^{d-1}$  to  $10^d - 1$ ) and test their digit sums; **Time Complexity**  $\mathcal{O}(10^d \cdot d)$ , **Space Complexity**  $\mathcal{O}(1)$
- **Greedy (construct smallest):**
  - Check if its possible to find the number, if not return -1 ( $\mathcal{O}(1)$ )
  - Reserve 1 for the first digit ( $s \leftarrow s - 1$ ).
  - From the *rightmost* digit to the second, set each to  $\min(9, s)$ , decreasing  $s$  each time
  - Set the first digit to  $s + 1$ ; output the  $d$  digits

# Smallest Number with Given Digits and Digit Sum

- **Problem:** Given digits  $d$  and sum  $s$ , find the smallest  $d$ -digit number with digit-sum  $s$  (no leading zeros), or report  $-1$  if impossible
- **Example:**
  - Input:  $d = 3, s = 20$
  - Output: 299
  - Explanation:  $2 + 9 + 9 = 20$
- **Naive (brute force):** Check all  $d$ -digit numbers (from  $10^{d-1}$  to  $10^d - 1$ ) and test their digit sums; **Time Complexity**  $\mathcal{O}(10^d \cdot d)$ , **Space Complexity**  $\mathcal{O}(1)$
- **Greedy (construct smallest):**
  - Check if its possible to find the number, if not return -1 ( $\mathcal{O}(1)$ )
  - Reserve 1 for the first digit ( $s \leftarrow s - 1$ ).
  - From the *rightmost* digit to the second, set each to  $\min(9, s)$ , decreasing  $s$  each time
  - Set the first digit to  $s + 1$ ; output the  $d$  digits
- **Complexity (greedy):** **Time Complexity**  $\mathcal{O}(d)$ , **Space Complexity**  $\mathcal{O}(1)$

# Smallest Number with Given Digits and Digit Sum

- **Problem:** Given digits  $d$  and sum  $s$ , find the smallest  $d$ -digit number with digit-sum  $s$  (no leading zeros), or report  $-1$  if impossible
- **Example:**
  - Input:  $d = 3, s = 20$
  - Output: 299
  - Explanation:  $2 + 9 + 9 = 20$
- **Naive (brute force):** Check all  $d$ -digit numbers (from  $10^{d-1}$  to  $10^d - 1$ ) and test their digit sums; **Time Complexity**  $\mathcal{O}(10^d \cdot d)$ , **Space Complexity**  $\mathcal{O}(1)$
- **Greedy (construct smallest):**
  - Check if its possible to find the number, if not return -1 ( $\mathcal{O}(1)$ )
  - Reserve 1 for the first digit ( $s \leftarrow s - 1$ ).
  - From the *rightmost* digit to the second, set each to  $\min(9, s)$ , decreasing  $s$  each time
  - Set the first digit to  $s + 1$ ; output the  $d$  digits
- **Complexity (greedy):** **Time Complexity**  $\mathcal{O}(d)$ , **Space Complexity**  $\mathcal{O}(1)$
- **Application:** Smallest ID/PIN satisfying a digit-sum checksum constraint

# Smallest Number with Given Digits and Digit Sum

## Greedy Algorithm

Input: `int d, int s`

Output: Smallest Number with Given Digits and Digit Sum

Instructions:

1. If  $s > 9 * d$  or ( $s == 0$  and  $d > 1$ )  
Return -1
2.  $digits \leftarrow$  string containing  $d$  '0's
3.  $s \leftarrow s - 1$  #Reserve first digit to 1
4. For  $i$  from  $d - 1$  to 0 do
  - a. If  $s > 9$   
 $digits[i] \leftarrow '9'$   
 $s \leftarrow s - 9$
  - b. Else  
 $digits[i] \leftarrow str(s)$   
 $s \leftarrow 0$
5.  $digits[0] \leftarrow str(1 + s)$
6. Return  $digits$

## Advanced Problems

# Minimize the Maximum Difference Between Heights

- **Problem statement:** Given heights  $a[]$  and an integer  $k$ , increase or decrease each  $a_i$  by  $k$  (once each, no negatives) to *minimize*  $\max(a) - \min(a)$
- **Example:**
  - Input:  $a = [1, 5, 10, 15]$ ,  $k = 3$
  - Output: 8
  - Explanation: Adjust to  $[4, 8, 7, 12] \Rightarrow 12 - 4 = 8$

# Minimize the Maximum Difference Between Heights

- **Problem statement:** Given heights  $a[]$  and an integer  $k$ , increase or decrease each  $a_i$  by  $k$  (once each, no negatives) to *minimize*  $\max(a) - \min(a)$
- **Example:**
  - Input:  $a = [1, 5, 10, 15]$ ,  $k = 3$
  - Output: 8
  - Explanation: Adjust to  $[4, 8, 7, 12] \Rightarrow 12 - 4 = 8$
- **Naive (brute force):** Try all  $2^n$  *increase/decrease* choices and take the best; **Time Complexity**  $\mathcal{O}(2^n \cdot n)$

# Minimize the Maximum Difference Between Heights

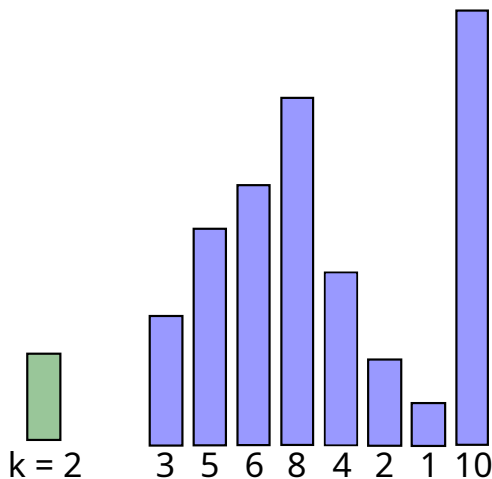
- **Problem statement:** Given heights  $a[]$  and an integer  $k$ , increase or decrease each  $a_i$  by  $k$  (once each, no negatives) to *minimize*  $\max(a) - \min(a)$
- **Example:**
  - Input:  $a = [1, 5, 10, 15]$ ,  $k = 3$
  - Output: 8
  - Explanation: Adjust to  $[4, 8, 7, 12] \Rightarrow 12 - 4 = 8$
- **Naive (brute force):** Try all  $2^n$  *increase/decrease* choices and take the best; **Time Complexity**  $\mathcal{O}(2^n \cdot n)$
- **Greedy):**
  - Sort  $a$ ; initial answer  $\text{ans} = a_n - a_1$
  - Set  $\text{small} = a_1 + k$ ,  $\text{big} = a_n - k$  (swap if  $\text{small} > \text{big}$ )
  - Define answer assuming no  $K$ :  $\text{res} = a[n-1] - a[0]$
  - For each element  $a_i$ , find smallest height  $\min(a[0] + k, a[i] - k)$  and tallest height  $\max(a[i-1] + k, a[n-1] - k)$ , then set  $\text{res}$  to smallest between current value of  $\text{res}$  and difference between tallest and smallest



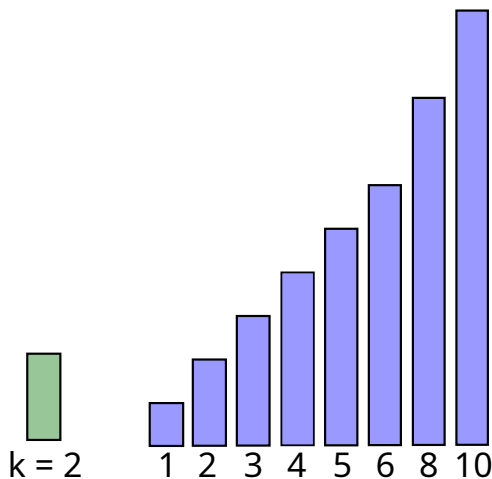
# Minimize the Maximum Difference Between Heights

- **Problem statement:** Given heights  $a[]$  and an integer  $k$ , increase or decrease each  $a_i$  by  $k$  (once each, no negatives) to *minimize*  $\max(a) - \min(a)$
- **Example:**
  - Input:  $a = [1, 5, 10, 15]$ ,  $k = 3$
  - Output: 8
  - Explanation: Adjust to  $[4, 8, 7, 12] \Rightarrow 12 - 4 = 8$
- **Naive (brute force):** Try all  $2^n$  *increase/decrease* choices and take the best; **Time Complexity**  $\mathcal{O}(2^n \cdot n)$
- **Greedy):**
  - Sort  $a$ ; initial answer  $\text{ans} = a_n - a_1$
  - Set  $\text{small} = a_1 + k$ ,  $\text{big} = a_n - k$  (swap if  $\text{small} > \text{big}$ )
  - Define answer assuming no  $K$ :  $\text{res} = a[n - 1] - a[0]$
  - For each element  $a_i$ , find smallest height  $\min(a[0] + k, a[i] - k)$  and tallest height  $\max(a[i - 1] + k, a[n - 1] - k)$ , then set  $\text{res}$  to smallest between current value of  $\text{res}$  and difference between tallest and smallest
- **Complexity:** **Time Complexity**  $\mathcal{O}(n \log n)$ , **Space Complexity**  $\mathcal{O}(1)$
- **Application:** Normalize variations (tower heights or product sizes) with limited adjustment budget  $\pm k$ .

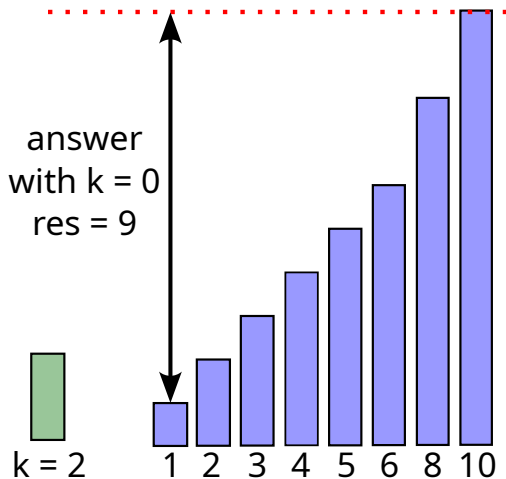
# Minimize the Maximum Difference Between Heights



# Minimize the Maximum Difference Between Heights

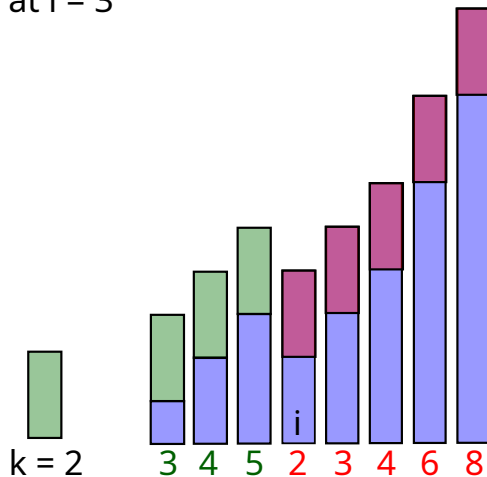


# Minimize the Maximum Difference Between Heights



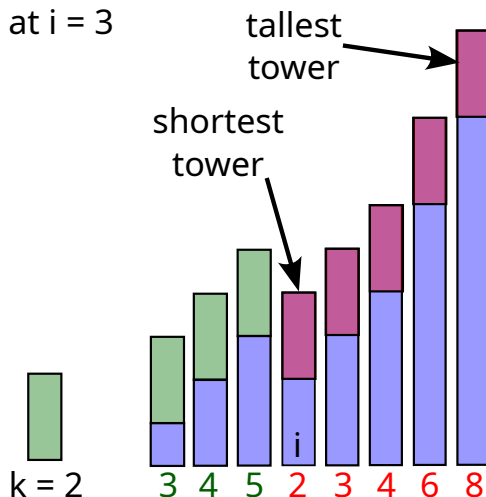
# Minimize the Maximum Difference Between Heights

at  $i = 3$



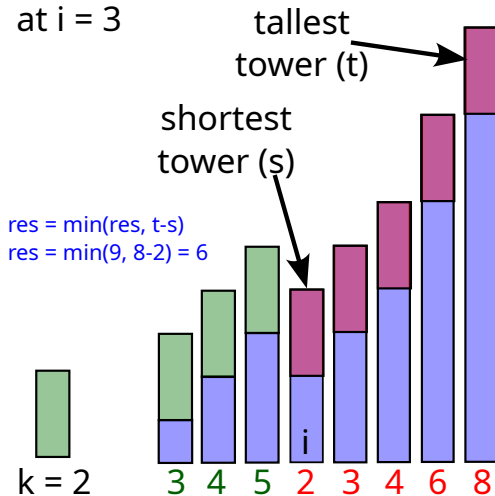
# Minimize the Maximum Difference Between Heights

at  $i = 3$



# Minimize the Maximum Difference Between Heights

at  $i = 3$



# Minimize the Maximum Difference Between Heights

## Greedy Algorithm

Input: Array  $A$  of size  $n$ , int  $k$

Output: Smallest Maximum Difference Between Heights After Adjustment

Instructions:

1. Sort  $A$  in increasing order
2.  $res \leftarrow A[n - 1] - A[0]$
3. For  $i$  from 1 to  $n - 1$  do
  - a. If  $A[i] - k < 0$   
     continue
  - b.  $smallest\_height \leftarrow \min(A[0] + k, A[i] - k)$
  - c.  $biggest\_height \leftarrow \max(A[i - 1] + k, A[n - 1] - k)$
  - d.  $res \leftarrow \min(res, biggest\_height - smallest\_height)$
4. Return  $res$